

Expected Paragraphs

What did you do?

In this phase of the project, I focused on making RetailSync feel more like a complete product instead of a collection of strong but separate workflows. Earlier work had already established authentication, statement processing, artifact review, and QuickBooks operations. In this report period, I worked more on tying those pieces together through product-level permissions, improved navigation, stronger review workflows, and a clearer operational demo path.

One of the biggest areas of progress was role-based access and product capability control. I expanded the RBAC system so workspace visibility and actions are driven more clearly by backend permission gates and frontend route protection. That work also supported a stronger Access workspace, where user management, invite handling, role editing, and permission assignment now feel more like part of the real product instead of simple admin utilities.

I also made meaningful progress in two major operational workflows. On the POS side, I added a Georgia monthly sales tax review workspace that calculates state and county tax obligations from existing daily POS data and presents the results in a structured review flow. On the accounting side, I continued improving the statement review and QuickBooks posting workflow, especially around deposits, check approval linking, posting direction, and review clarity. Overall, this phase was about product integration, permission-aware UX, and making the strongest demo flows easier to explain and use.

How did you do it?

I continued building the project with the same full-stack TypeScript architecture. The frontend uses React, Redux Toolkit, and Material UI, while the backend uses Express, MongoDB, Mongoose, and Zod. The monorepo structure remained important because shared schemas, module constants, and permission definitions are now reused across more parts of the system than before.

My workflow in this phase was more integration-driven than before. Instead of only building isolated features, I spent more time tracing how a real user moves through the application: sign in, land on a dashboard, see only the workspaces they are allowed to access, manage users or roles if permitted, review POS tax data, open statement review, and continue into QuickBooks-related workflows. That meant implementing backend permission gates, aligning frontend navigation and page access, and then filling in the workflow details that make those screens useful.

I also relied on documentation, test coverage, and workflow-specific utilities to keep the project manageable. For the POS sales tax workspace, I built client-side aggregation and calculation logic on top of existing daily records. For the accounting workflow, I continued using shared schemas, statement-processing services, review helpers, and QuickBooks integration layers to support more reliable posting and review behavior. This phase involved a lot of iteration because changes to permissions, UI states, and review workflows tend to affect multiple modules at once.

What problems did you encounter?

One of the main challenges was keeping the permission model both flexible and understandable. It is not enough to hide a page in the frontend; the backend also needs to enforce the same access rules, and the user experience needs to explain why something is available, locked, or hidden. That made RBAC work more complex because it touched routing, API middleware, settings, user management, dashboard presentation, and workspace-level actions all at once.

Another challenge was making the product feel cohesive while still respecting the real complexity of the workflows. The statement review path, QuickBooks operations, user administration, and POS tax review all solve different problems, but they still need to feel like part of one application. That required more attention to navigation, product surfaces, locked states, access boundaries, and how one workflow leads into another without confusing the user.

I also encountered challenges in the accounting workflow itself, especially around statement review correctness. Deposits, check approvals, posting direction, and bank-account mapping all need to line up correctly for the workflow to feel trustworthy. Since these features connect parsed statement data with downstream accounting actions, I had to be careful about shared schema changes, test coverage, and edge cases where partial review data could create confusing results.

What Did You Do

What features did you finish?

I finished or significantly improved the product-capability RBAC system, including backend permission gates, route protection, permission-aware dashboard sections, role management improvements, and stronger Access workspace behavior for members, invites, and role administration. This made the product shell more realistic because users now see and use the application according to their actual permissions rather than through mostly static navigation.

I also finished the Georgia monthly sales tax review workspace in the POS module. That feature uses existing daily POS data to generate a monthly tax review experience with year and month selection, tax breakdowns, vendor compensation calculations, payable sales tax summaries, and supporting daily-record drill-down. In addition, I continued improving the accounting statement review and QuickBooks posting workflow with better deposit handling, check approval linking, bank-account mapping support, and more structured review behavior.

Can the features be demoed?

Yes, the features can be demoed. A demo can now show sign-in, dashboard entry with permission-aware workspace visibility, opening the Access hub to manage users and roles, switching into the POS workspace to review Georgia monthly sales tax, and then moving into the accounting statements workspace to review statement processing and its QuickBooks-related downstream context. This makes the overall product easier to present as one connected system rather than a few unrelated modules.

How many of the features are integrated in a way where they can be used together?

A large portion of the important features are now integrated in a way where they can be used together. Authentication, onboarding, product-level permissions, dashboard navigation, user and role management, POS reporting, Georgia sales tax review, statement review, and QuickBooks operations now connect more clearly inside one product shell. Some areas still need additional hardening and broader release validation, but the project now behaves much more like a unified platform with defined workspaces and controlled access.

How Did You Do It

Did you use any project templates or tutorials?

I did not use a full project template, but I followed common patterns from React, Express, MongoDB, TypeScript, and permission-based application design. I also used official documentation and reference material for areas such as route protection, API authorization, testing, QuickBooks integration behavior, and data-driven UI workflows.

Did you seek help from the instructor or other colleagues?

I mainly implemented the features independently, but I did use outside input when thinking through product scope, architecture, workflow design, and how to present the strongest integrated demo path. That feedback was especially useful in helping me focus on making the active modules more cohesive instead of continuing to expand the project in too many directions at once.

Did you use any AI/ML/LLMs?

Yes, I used AI tools during development. I used them for debugging support, implementation planning, refactoring help, documentation drafting, and understanding integration issues more quickly. However, I still verified the actual behavior through code changes, tests, and project-specific review before treating anything as complete.

What Problems Did You Encounter

Do you have any concerns about your progress?

My main concern is still scope control and release confidence. RetailSync now has stronger integration across dashboard access, roles, POS workflows, statements, and QuickBooks operations, but that also increases the number of cross-module interactions that need to stay stable. Because of that, I need to keep prioritizing test coverage, documentation, and a reliable end-to-end demo path instead of continuing to add major new product areas.

I am also concerned about the difference between feature completeness and release readiness. Many important surfaces are implemented and demoable, but some still depend on environment-sensitive integrations, broader regression coverage, or additional UX hardening. The project has reached a stage where the most important work is not only adding features, but also making sure the current integrated experience is dependable and easy to evaluate.

Do you need any help currently?

The most useful help right now would be feedback on the final presentation and which integrated workflows should be emphasized most during evaluation. I would also benefit from feedback on whether the strongest grading story should focus more on the permission-aware product shell, the POS sales tax workflow, or the statement-to-QuickBooks accounting flow.